

Progress Report 1

Enrique Campos, Daniela Blanco del Real, Joaquín Mir, Patricia Renart

April 2024

1 Problem description, current hypotheses and preliminary solutions

Our project aims to explore and study how various weight initialization methods impact the performance of Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) models. Common problems of RNNs include vanishing gradients, exploding gradients, slow convergence or symmetry learning

- **Amplification of the Vanishing/Exploding Gradient Problems**

Vanishing gradients can become more prevalent if the initial weights are too small, leading to gradients that diminish too quickly as they are propagated back through time, thereby slowing down or halting the learning process.

Exploding gradients are more likely if the initial weights are too large, causing gradients to grow exponentially during backpropagation and potentially leading to numerical instability or divergence of the model parameters.

- **Slow Convergence**

Bad weight initialization can lead to a situation where the optimization algorithm struggles to find a good direction to minimize the loss function, resulting in slow convergence or convergence to suboptimal minima.

- **Symmetry Breaking Problems**

In RNNs, particularly when all weights are initialized to the same value, neurons can end up learning the same features during training. This lack of diversity in the learning process can significantly impede the network's capacity to capture complex patterns in the data.

2 Current hypotheses and preliminary results

As it has been previously stated, vanishing and exploding gradient issues, poor convergence, and symmetry breaking are critical challenges in training Recurrent Neural Networks (RNNs) and LSTM, significantly influenced by the choice of weight initialization strategies. Therefore, our initial hypotheses and preliminary ideas/results, which will be proved in further stages of the project, would be:

Traditional methods, such as initializing weights to a constant value, can lead to symmetry breaking problems, as neurons fail to learn diverse features. Initializing weights to zero is particularly detrimental, as it prevents the network from learning anything at all.

On the other hand, using an identity matrix for initialization can somewhat mitigate the vanishing gradient problem by maintaining gradient flow, whereas scaling the identity matrix by a factor of 0.01 can address both vanishing and exploding gradients by balancing the scale of weight updates.

Uniform, normal and truncated random initialization helps avoid symmetry breaking, thereby aiding convergence by ensuring that neurons start in different states.

Xavier (Glorot) and Kaiming (He) initialization methods tackle the vanishing and exploding gradient problems by aiming to maintain the variance of weights across layers, facilitating a more stable and efficient training process.

Moreover, orthogonal initialization leverages the algebraic property of having eigenvalues of one to prevent both vanishing and exploding gradients, promoting a more robust learning environment for RNNs by preserving gradient magnitude over time.

3 First implementations

This part of our report will cover the explanation for the basic LSTM model that has been implemented in order to fulfill our initial milestones. Both the data processing and the model itself will be explained, while the implementations of several initialization techniques will be covered further on in our project.

Data processing

To make a base model for the LSTM network we have decided to use part of the MAESTRO dataset. It's application has been fairly easy as we wanted to treat music as NLP, an approach that's shown to be quite successful.

However, our initial idea in processing the data by saving the note pitch, octave and duration type only worked for melodic music. Therefore, we decided to follow an approach taken in several research papers where music is processed by events. For example, saving the moment a note starts or ends, velocity or the time between events. Its implementation has been fairly easy thanks to the *muspy* library, developed to help processing different types of music files, as it also gives already each event as an integer.

Embedding model

The next step has been to design the net. We decided to use first an **embedding model** so different words (or events) would be related between each other. After that, we would pass those relations through the LSTM network and get a melody prediction. Some challenges arose in these steps when implementing the model as dimensions needed to match. However, after some debugging, we were able to determine the correct parameters in each step for it to train correctly.

LSTM model and current stage

The last step of this implementation process was to create our project's architecture and the LSTM model itself. We worked on dividing tasks between files, using them as function libraries all topic-related within each file instead of creating multiple operative files.

Our LSTM model depends on the existence of a pre-trained embedding model located in the */runs* directory. Once this is checked, the model training can be easily initialized by running our *main.py*, which will start the train and validation loop.

We have decided to use an extremely simple LSTM model so that the effect of the different techniques can be easily distinguished from one another. Up to this point, our implementations have focused on preparing this model for optimal functioning and assuring its strength. Nevertheless, this optimal functioning has already been achieved, allowing us to **start implementing the different initialization techniques shortly**.

4 Planning

As stated in the project proposal, our intentions towards this first week of progress were clear. This part of the project was intended to be executed separately, dividing our members in pairs and each focusing on different tasks. The first pair would have to gather the mathematical knowledge, whilst the other would be in charge of creating a first LSTM model implementation working with a new dataset (polymorphic music modelling).

As for our upcoming actions, the following guidelines have been set for next week's milestone achievement:

1. Achieve a better pre-trained embedding model for our LSTM
2. Create a .py that gathers the **algorithmic implementations** of all the initialization techniques
3. Adapt previous RNN implementations to our current architecture
4. Create a general approach so that the implementations file can be used for both RNN and LSTM models
5. Create graphical examples showing the differences between the initializations